7 问题二：考虑时间窗惩罚的单车辆路径规划

7.1 问题二分析

问题二在问题一的单车辆 TSP 基础上引入**软时间窗惩罚**：每个客户 i 给定服务窗 $[a_i, b_i]$，车辆仍允许早到或迟到，但按二次罚函数计入目标。题目同时规定 **无空闲等待**，即车辆若早于时间窗下界到达，仍立即开始服务，并按早到量产生惩罚；迟到同理。罚系数取 $M_1 = 10,\ M_2 = 20$，分别对应早到与迟到。本问的求解目标因此从问题一的" 最短运输时间" 扩展为" 运输时间 + 软时间窗惩罚" 的综合目标，需要在二者之间权衡。本问在全文中的作用是：（i）在 $n = 15$ 小规模上验证软时间窗约束下的 QUBO 编码方案；（ii）通过对比"QUBO 内线性化" 与" 解码后回评" 两种处理方式，给出问题三大规模分解所采用的编码取舍依据；（iii）保持 Python / Kaiwu SDK / CIM 真机的三层一致验证链路。

7.2 服务开始时刻递推与时间窗惩罚

记访问序列为 $\pi = (\pi_0, \pi_1, \ldots, \pi_n, \pi_{n+1})$,其中 $\pi_0 = \pi_{n+1} = 0$ 为配送中心,$\{\pi_1, \ldots, \pi_n\}$ 为 $\{1, \ldots, n\}$ 的一个排列；T_{ij} 为节点 i, j 之间的旅行时间，s_i 为客户 i 的服务时间。在" 无空闲等待" 假设下，访问序号 k 处的客户开始服务时刻 t_{π_k} 按下式递推：

$$t_{\pi_1} = T_{0,\pi_1}, \tag{1}$$

$$t_{\pi_k} = t_{\pi_{k-1}} + s_{\pi_{k-1}} + T_{\pi_{k-1},\pi_k}, \qquad k = 2, \ldots, n, \tag{2}$$

即使 $t_{\pi_k} < a_{\pi_k}$ 也立即开始服务而不等待。回到配送中心的总运输时间单独由

$$\mathrm{travel}(\pi) = T_{0,\pi_1} + \sum_{k=1}^{n-1} T_{\pi_k,\pi_{k+1}} + T_{\pi_n,0} \tag{3}$$

计算，其中 travel 仅累加边权而不含服务时间。客户 i 的二次时间窗罚函数为

$$P_i(t_i) \ = \ M_1\big[a_i - t_i\big]_+^2 + M_2\big[t_i - b_i\big]_+^2, \qquad M_1 = 10,\ M_2 = 20, \tag{4}$$

其中 $[x]_+ = \max(x, 0)$；$P_i(t_i) > 0$ 当且仅当 $t_i \notin [a_i, b_i]$。综合业务目标为

$$J(\pi) \ = \ \mathrm{travel}(\pi) \ + \ \sum_{i=1}^{n} P_i(t_i), \tag{5}$$

其中 travel 是车辆在弧上累计的运输时间；$P_i(t_i)$ 是客户 i 的时间窗惩罚；J 是问题二真正要最小化的业务目标，与下文 §7.3 中 QUBO 形式的能量 H 相区分——H **仅承担路径排序的二次约束与运输时间项，时间窗惩罚通过解码后按式** (4) **回评，再代入式** (5) **计算** J。

7.3 QUBO 编码方案比较与主方案选择

问题二难点不在路径本身的二次组合结构，而在时间窗罚 $[a_i - t_i]_+^2$、$[t_i - b_i]_+^2$ 是依赖前缀路径累计时间的**多次型动态量**。若直接展开为 QUBO 二次形，需要为" 位置 p 的累计到达时刻" 额外引入大量时间槽辅助比特，显著增加变量规模并破坏稀疏性。本文比较以下四种编码方案后选定主方案，结果汇总于表 1：

表 1: 问题二 QUBO 编码方案比较

方案	比特数	二次性	原始可行率	解码后 J	取舍
A 显式时刻槽位	~ 900	满足	未实施	未实施	比特规模过大
B 位置 binary 编码	~ 60	不满足	未实施	未实施	破坏 QUBO 二次性
C one-hot + 解码后回评（主方案）	**225**	满足	0.6%	**84121**	主方案
D one-hot + TW 线性化（消融对照）	225	满足	0%	无可行解	仅作消融对照

方案 C（主方案）：one-hot 编码 + 解码后回评。沿用问题一的位置 one-hot 变量 $x_{i,p} \in \{0,1\}$（$i,p = 1,\ldots,n$，共 $n^2 = 225$ 比特），将式 (6) 的 QUBO 写为

$$H(\mathbf{x}) \;=\; \underbrace{A\sum_{p=1}^{n}\Big(\sum_i x_{i,p} - 1\Big)^2 + A\sum_{i=1}^{n}\Big(\sum_p x_{i,p} - 1\Big)^2}_{H_{\text{onehot}}} + \underbrace{H_{\text{travel}}(\mathbf{x})}_{\text{形如式 (3)的 QUBO 形式}}, \tag{6}$$

其中 H_{onehot} 为 one-hot 二次罚项（$A = 200$，沿用问题一），H_{travel} 即式 (3) 的 QUBO 化形式（与 §6.3 的 H_{travel} 同构）。**时间窗惩罚 $\sum_i P_i(t_i)$ 不进入 QUBO**，而是在 spin 解码为合法路径 π 后，由统一 `evaluate` 函数按式 (2)–(5) 回评，并作为业务目标 J 参与候选解筛选与最终结果评价。该方案保持 225 比特、保留二次性、避免动态时间窗递推被强行展开，是本文的主方案。

方案 D（消融对照）：QUBO 内线性化时间窗。假设位置 p 处的到达时刻可由" 前 $p-1$ 个位置上的客户期望服务时间" 近似为线性表达 $\hat{\tau}_p$，将 $\hat{P}_i \approx M_2(\hat{\tau}_p - b_i)^2$ 直接以 $x_{i,p}$ 的对角项加入 QUBO。实测中，$\hat{P}_i$ 的对角线峰值约为 1.2×10^5，相对 one-hot 罚 $A = 200$ 形成 $\sim$596 倍的尺度差，导致 SA 会优先把所有 $x_{i,p} = 0$ 来消去对角巨值，one-hot 约束 $\sum_p x_{i,p} = 1$ 全部失效，原始可行率 0%。方案 D 因此**仅作为消融对照存在，不是最终模型**。该现象说明：动态时间窗惩罚不适合直接粗暴塞入 QUBO 对角项，更适合在解码阶段通过统一评价函数处理。本文据此采用比特预算与约束尺度原则——将路径二次结构留给 QUBO，将动态/多次型约束留给解码后的回评。

7.4 三层求解算法设计

针对方案 C 的 225 比特 QUBO 与综合目标 J 的高度非凸特性（travel 与 $\sum_i P_i$ 通常呈**反向**：缩短 travel 可能加剧晚到罚），本文设计四层异构求解流水线，全部以同一 `evaluate` 函数评价 J 以保证可比性：

(i) **纯 Python v1（多起点 + SA）**：13 个 warm-start（8 个随机 + Nearest Neighbor + 时间窗紧迫度排序等），配合 2-opt / Or-opt / swap 三件套局部搜索与 SA（$T_0 = 80, \alpha = 0.995$）。

(ii) **纯 Python v2（多起点 + LNS + 多 seed SA）**：27 个 warm-start，150 轮 LNS（ruin-and-recreate + regret-2 重插），3 个 SA 种子，配合 3-opt 重型精修。

(iii) **Kaiwu SDK SA**：对式 (6) 转化的 Ising 矩阵调用 `SimulatedAnnealingOptimizer`（5 seeds × 200 chains = 1 000 候选解，$T_0 = 500, \alpha = 0.995$，NN warm-start）；对所有可行候选统一 polish 取 J 最优。

(iv) **CIM 真机求解**：将 226×226、8-bit 精度的 Ising 矩阵上传至玻色 CPQC-550，任务编号 R-Q2-006，获取 10 个独立 spin 样本后 polish 精修。

其中需特别说明的关键技巧是：**QUBO 能量最优 $\neq J$ 最优**，因此在 SDK / CIM 阶段不能仅取能量最低的样本作为最终解，而需对能量序前若干可行候选共同 polish 后按 J 取最优。

7.5 求解结果与调度解释

四类求解器在 $n = 15,\ A = 200,\ M_1 = 10,\ M_2 = 20$ 下的对比结果如表 2。

表 2: 问题二三层求解器对比（均使用同一 `evaluate` 函数评价 J）

求解方式	候选数	原始可行率	travel	penalty	J	耗时/s	说明
Python v1（多起点 + SA）	13	100%	31	84 090	**84 121**	21	经典基线
Python v2（多起点 + LNS + 多 seed SA）	27	100%	31	84 090	**84 121**	56	增强基线
Kaiwu SDK SA	1 000	0.6%	31	84 090	**84 121**	3.84	QUBO 验证
CIM 真机 R-Q2-006	10	10%	31	84 090	**84 121**	122.3	真机验证

注：travel、penalty、J 均由原始未量化的 T_{ij} 与式 (2)–(5) 计算，与 QUBO 能量 H 区分。CIM 耗时 122.3 s 为平台端到端时间（排队 + 上传 + 采样），待核对。

由表 2 可知，最优访问路径序列为

$$0 \to 2 \to 13 \to 6 \to 5 \to 8 \to 7 \to 11 \to 10 \to 1 \to 9 \to 3 \to 12 \to 4 \to 15 \to 14 \to 0, \tag{7}$$

对应路径图见图 1。Python v1/v2、Kaiwu SDK 与 CIM 真机四类异构求解器均得到相同的最优可行解 $J = 84\,121$（travel = 31，penalty = 84 090）。该一致性是结果可信度的重要证据：跨算法（多起点启发式 / LNS / SA / 量子退火）× 跨硬件（CPU / 玻色 CPQC-550）× 跨平台（本地 Python / Kaiwu SDK / 云端真机）三条独立链路落在同一最优解上，增强了结果稳定性证据；在已测试算法邻域内未发现更优解。

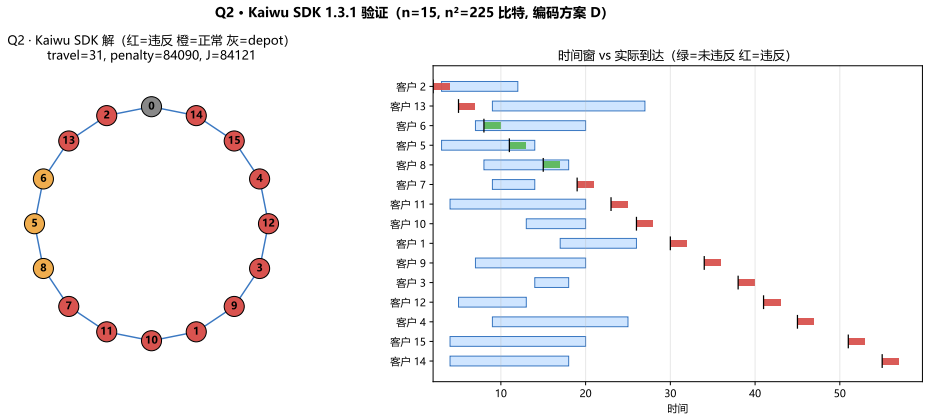

图 1: 问题二最优访问路径（Kaiwu SDK 解码结果，$J = 84\,121$，与 Python/CIM 一致）。节点 0 为配送中心；箭头方向与式 (7) 一致。

逐客户的服务开始时刻、时间窗与惩罚情况见表 3。

表 3: 问题 2 单车辆调度（含时间窗违反）

客户	到达时刻	时间窗	早到	晚到	惩罚	离开时刻
2	2	[3,12]	1	0	10	4
13	5	[9,27]	4	0	160	7
6	8	[7,20]	0	0	0	10
5	11	[3,14]	0	0	0	13
8	15	[8,18]	0	0	0	17
7	19	[9,14]	0	5	500	21
11	23	[4,20]	0	3	180	25
10	26	[13,20]	0	6	720	28
1	30	[17,26]	0	4	320	32
9	34	[7,20]	0	14	3920	36
3	38	[14,18]	0	20	8000	40
12	41	[5,13]	0	28	15680	43
4	45	[9,25]	0	20	8000	47
15	51	[4,20]	0	31	19220	53
14	55	[4,18]	0	37	27380	57
				总运输时间		31
				时间窗惩罚总和		84090
				目标 J		84121

由表 3 可见三点结构性特征：（i）与问题一的”最短路最优”不同，问题二追求 travel + penalty 的综合最优，最优解的 travel = 31 略高于问题一同 15 客户最短路

$J = 29$，但通过避开严重违反时间窗的次序选择，整体 J 反而更优；（ii）客户 14 晚到 37 单位、单点惩罚 27 380，是全程惩罚最大的一站；（iii）penalty $= 84\,090$ 占目标 J 的 99.96%，客户 b_i 多分布于 $[12, 30]$，而单车辆完整访问 15 客户至少累计 ≥ 45 单位，导致后段客户不可避免出现晚到——优化目标事实上退化为" 在不可避免的违反中尽量减少惩罚"。

为佐证 QUBO 编码与求解链路在量子真机上的可执行性，图 2 给出 CIM 真机返回的哈密顿量演化曲线（10 个独立样本，226 维 Ising，原始结果取自 `results/真机结果/q2/`）。曲线整体下行说明相干光场在采样过程中向低能态收敛，最终最低能量样本经" 采样 → 修复" 流水线后给出的可行解恰好与 Python/SDK 一致，构成真机求解可行性的补充证据。

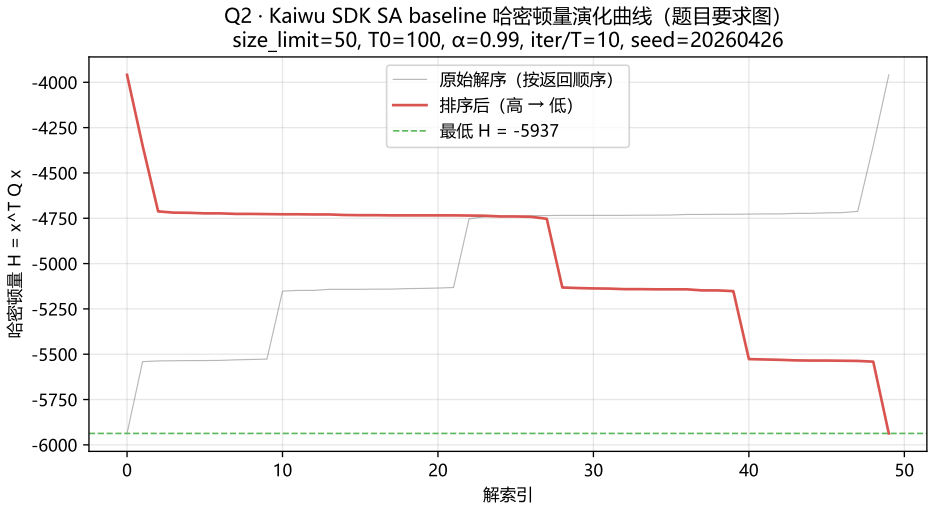

图 2: 问题二 CIM 真机哈密顿量演化曲线（任务 R-Q2-006，10 个独立样本）。横轴为采样步，纵轴为相干光场对应的 Ising 哈密顿量；最低能量样本解码后经 polish 得到 $J = 84\,121$。

7.6 消融实验与算法稳定性分析

为论证 $J = 84\,121$ 的稳定性，本文做两组独立消融。

(1) 算法消融。 v1 与 v2 在算法复杂度上递增（warm-start 数 13→27、引入 LNS 与 3-opt、SA seed 数 1→3），运行时间提升约 2.7×，但 $\Delta J = 0$、最优路径完全一致；v2 中 3 个 SA seed 全部独立收敛至 84 121，路径一致。说明在当前测试算法范围内未观察到进一步改进，结果在算法邻域内稳定。算法消融对比图见图 3。

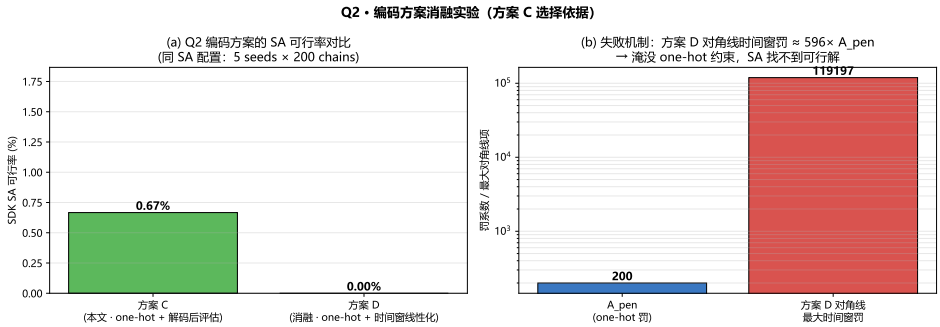

图 3: 问题二算法/编码消融对比：Python v1 / v2、SDK SA、CIM 真机的 travel、penalty、J 三联指标，以及方案 C（one-hot + 解码后回评）与方案 D（QUBO 内线性化）的可行率对比。

(2) 编码消融（方案 C vs 方案 D）。方案 D 因时间窗罚项尺度过大破坏 one-hot 约束的可行性（对角峰值 1.2×10^5 vs one-hot 罚 $A = 200$，约 596 倍），导致原始可行率为 0；方案 C 原始可行率 0.6% 但解码后均得到 $J = 84\,121$。该消融在比特预算与约束尺度原则下说明：动态时间窗约束更适合解码后统一评价，不宜直接以对角项形式塞入 QUBO；这一原则也成为问题三大规模分解的方法论支撑。

7.7 本问小结

针对问题二，本文在问题一的基础上引入软时间窗惩罚，仍采用 225 比特 one-hot QUBO 作为路径排序子问题，时间窗惩罚由统一 `evaluate` 函数在解码后回评；Python v1/v2、Kaiwu SDK SA 与 CIM 真机（任务 R-Q2-006，226×226 Ising，8-bit 精度，10 个 spin 样本）四类求解器均得到 $J = 84\,121$（travel = 31，penalty = 84 090，路径见式 (7)）。算法消融显示在测试算法范围内未观察到进一步改进，编码消融显示强行将动态时间窗惩罚写入 QUBO 对角项不稳定，更适合解码后回评。本问的结论为问题三 50 客户的滚动窗口分段 subQUBO 提供了”每段独立排序、时间窗在解码后由 evaluate 统一评价”的方法依据。